UNIT – 3
Variables and Data Types

Compiled By: Er. Krishna Khadka

Fundamental of C

- Program is a set of instructions to interact with computer.
- The programs are written in programing language.
- C is a procedural programming language developed at AT & T's Bell Laboratories of USA in 1972.
- It was designed and written by a man named Dennis Ritchie.
- C program is a sequence of characters that will be converted by the C compiler into machine code.

Character set of C

- Character set is a package of valid characters recognized by compiler/interpreter.
- Each natural language has its own alphabet and characters set as the basic building.
- We combine different alphabets to make a word, sentence, paragraph and a meaningful passage.
- Similarly each computer languages have their own character set.
- Particularly, C also has its own character set, shows in following table:

Uppercase alphabets	A, B, C.....Z
Lowercase alphabets	a, b, c.....z
Digits	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Special symbols	+ - * / ! ? | \ < > & , : ; # % * ( ) [ ] . _ $ ^
White space characters	Blank space, new line, horizontal tab, vertical tab, etc

Keywords

- Keywords are the predefined reserved (built-in) words whose meaning has already been defined to the c compiler.
- A programmer can't use the keywords for other task than the task for which it has been defined.
- For example: a programmer can't use the keyword for variable names.
- There are 32 keywords in c, which are listed in figure:

auto	do	goto	sizeof
break	double	if	static
case	else	int	switch
char	union	long	typedef
struct	enum	register	unsigned
const	extern	return	void
default	float	short	volatile
continue	for	signed	while

Identifiers

- Identifiers are the user defined names and consists of a sequence of letters and digits with letter or underscore as a first character.
- It may be name of the variables, functions, arrays, etc.
- In identifiers we can use both upper and lower cases.

Rules for Identifiers

- First characters must be an alphabet or underscore.
- Must consist of only alphabets, digits or underscore.
- Cannot use a keyword.
- Must not contain white spaces.
- To connect two words underscore can be used.

Examples for valid or invalid identifier

a. record1

→This is valid identifier.

b. file_3

→This is valid identifier.

c. #tax

→Invalid identifier because identifier must begin with a letter and special character like # is not permitted in identifiers.

d. goto

→Invalid identifier because keywords cannot be used as an identifier.

e. Name-and-address

→Invalid identifier because hyphen or dash is not permitted.

f. void

→Invalid identifier because keywords cannot be used as an identifier

Constants

- Constant in C refers to fixed values that do not change during execution of a program.
- C supports several types of constants, such as numeric constants and character constants.

Numeric constants: It consists of:

Integer Constants:

- It refers to the sequence of digits with no decimal points. The three kinds of integer constants are:

1. Decimals:

Decimal numbers are set of digits from 0 to 9 with leading +ve or –ve sign.

For example: 121, -512 etc.

2. Octals:

Octal numbers are any combination of digits from set 0 to 7 with leading 0.

For example: 0121, 0375 etc.

3. Hexadecimals:

Hexadecimal numbers are the sequence of digits 0-9 and alphabets a-f (A-F) with preceding 0X or 0x.

For example: 0xAB23, 0X787 etc.

Real or Floating point Constants:

- They are the numeric constants with decimal points.
- The real constants are further categorized as:

Fractional Real constant:

- They are the set of digits from 0 to 9 with decimal points. For example: 394.7867, 0.78676 etc.

Exponential Real constants:

- In the exponential form, the constants are represented in two form:
  Mantissa E exponent;
- Where, the Mantissa is either integer or real constant but the exponent is always in integer form.
  - For example: 21565.32 = **2.156532** E**4**, where E4 = 10^4.

Character constant:

- Character constant are the set of alphabets.
- Every character has some integer value known as American Standard Code for Information Interchange (ASCII).
- **The character constants are further categories as:**

Single character constant:

- It contains a single character enclosed with in a pair of single quote mark (‘ ’).
- Thus, ‘X’, ‘5’ are characters but X, 5 are not.

String constant:

- String constants are a sequence of characters enclosed in double quotes marks (“ ”). They may be letters, numbers, special symbols or blank spaces.
- For example: “Hello”, “X+Y”. Note: ‘x’ ≠ “x” .

Characters	**ASCII Values**
A – Z	(65 – 90)
a - z	(97 – 122)
0 - 9	(48 – 57)

Delimiters

- Delimiters are small system components that separate the various elements (variables, constants, statements) of a program.
- They are also known as separators.

Some of the delimiters are:

- Comma: It is used to separate two or more variables, constants.
- Semicolon: It is used to indicate the end of a statement.
- Apostrophes (single quote): It is used to indicate character constant.
- Double quotes: It is used to indicate a string.
- White space: Space, tab, new line etc.

Example of Delimiters

```
#include<stdio.h>
void main()
{
int a,b,c[10];
label_num:
printf("hi");
goto label_num:
}
```

Delimiters	Symbols	Use
Colon	:	Useful for label
Semicolon	;	Terminates statements
Parenthesis	()	Used in Expression and function
Square brackets	[]	used for array declaration
Curly braces	{}	Scope of statement
Hash	#	Preprocessor directive
Comma	,	variable seperator

Variables

- A variable is a data name that may be used to store a data value.
- Unlike constants that remain unchanged during the execution of a program, a variable may take different values at different times during execution.
- Constant supplied by the user is stored in block of memory by the help of variable.
- Consider an expression:
  `2*x + 3*y = 5.`

  In this expression the quantities x and y are variable.

  A variable is a named location in memory that is used to hold certain values.

Rules for declaring variable name

1. Every variable name in C must start with a letter or underscore.
2. A valid variable name should not contain commas or blanks.
3. Valid variable name should not be keyword.
4. A variable name must be declared before using it.

Declaring variable

- After defining suitable variable name, we must declare them before used in our program.
- The declaration deals with two things:

- It tells the compiler what the variable name is.

- It specifies what type of data the variable can hold.

Syntax:

- Data_type variable_names;

For example:

- int account_number;
- float a, b, c;
- char name[10];

Data Types

- Data types are used to declare the variables and tell the compiler what types of data the variable will hold.
- Data type is useful in C that tells the computer about the type and nature of the value to be stored in a variable.
- The type of a variable determines how much space it occupies in storage.
- Generally the data types can be categorized as:

✓ **Primary Data Types**

✓ **Derived (or Secondary) Data Types**

✓ **User Defined Data Types**

Primary Data Types

- These are the basic data types.
- There are three basic data types in C.
- They are integer data type, real or floating data type and void type.

Data type	Description	Required memory
char	Character	1 Byte
int	Integer	2 Byte
float	Floating point number	4 Bytes
double	Double precision floating point number	8 Bytes
void	No value	0 bytes

Derived (or Secondary) Data Types

- The secondary (derived) data types are a collection of primary (primitive) data types.
- These are derived from the primary data types.
- Example: array, union, structure, etc

User Defined Data Types

- The data types defined by the user is known as user defined data types.
- Example: enum(enumerated data type) and typedef (type definition data type).

Operators

- C supports a rich set of operators.
- An operator is a symbol that tells the computer to perform mathematical or logical operations on operands.
- Operators are used in programs to manipulate data.
- C operators can be classified into a number of categories:

Arithmetic operators:

- Arithmetic operators are used for performing most common arithmetic (mathematical) operations.

Operator	Meaning of Operator
+	addition or unary plus
-	subtraction or unary minus
*	multiplication
/	division
%	remainder after division(modulo division)

```c
#include <stdio.h>
int main()
{
    int a = 9,b = 4, c;

    c = a+b;
    printf("a+b = %d \n",c);

    c = a-b;
    printf("a-b = %d \n",c);

    c = a*b;
    printf("a*b = %d \n",c);

    c=a/b;
    printf("a/b = %d \n",c);

    c=a%b;
    printf("Remainder when a divided by b = %d \n",c);

    return 0;
}
```

```
a+b = 13
a-b = 5
a*b = 36
a/b = 2
Remainder when a divided by b = 1
```

Logical operators

- Expressions which use logical operators are evaluated to either true or false.

Operator	Meaning of Operator	Example
&&	Logial AND. True only if all operands are true	If c = 5 and d = 2 then, expression `((c == 5) && (d > 5))` equals to 0.
||	Logical OR. True only if either one operand is true	If c = 5 and d = 2 then, expression `((c == 5) || (d > 5))` equals to 1.
!	Logical NOT. True only if the operand is 0	If c = 5 then, expression `! (c == 5)` equals to 0.

A	B	A&&B	A||B	!A
T	T	T	T	F
T	F	F	T	F
F	T	F	T	T
F	F	F	F	T

Table: Truth Table for Logical AND, OR and NOT

```c
// C Program to demonstrate the working of logical operators
#include <stdio.h>
int main()
{
    int a = 5, b = 5, c = 10, result;

    result = (a == b) && (c > b);
    printf("(a == b) && (c > b) equals to %d \n", result);

    result = (a == b) && (c < b);
    printf("(a == b) && (c < b) equals to %d \n", result);

    result = (a == b) || (c < b);
    printf("(a == b) || (c < b) equals to %d \n", result);

    result = (a != b) || (c < b);
    printf("(a != b) || (c < b) equals to %d \n", result);

    result = !(a != b);
    printf("!(a != b) equals to %d \n", result);

    result = !(a == b);
    printf("!(a == b) equals to %d \n", result);

    return 0;
}
```

```
(a == b) && (c > b) equals to 1
(a == b) && (c < b) equals to 0
(a == b) || (c < b) equals to 1
(a != b) || (c < b) equals to 0
!(a != b) equals to 1
!(a == b) equals to 0
```

Assignment Operator:

- An assignment operator assign a value to a variable and is represented by "=" (equals to) symbol.
- **Syntax:** Variable name = expression.
- The Compound assignment operators (+=, *=, -=, /=, %=) are also used.

Operator	Name	Example
=	Assignment	C = A + B will assign value of A + B into C
+=	Add AND assignment	C += A is equivalent to C = C + A
-=	Subtract AND assignment	C -= A is equivalent to C = C – A
*=	Multiply AND assignment	C *= A is equivalent to C = C * A
/=	Divide AND assignment	C /= A is equivalent to C = C / A
%=	Modulus AND assignment	C %= A is equivalent to C = C % A

```c
#include <stdio.h>
int main()
{
    int a = 5, c;
    c = a;
    printf("c = %d \n", c);
    c += a; // c = c+a
    printf("c = %d \n", c);
    c -= a; // c = c-a
    printf("c = %d \n", c);
    c *= a; // c = c*a
    printf("c = %d \n", c);
    c /= a; // c = c/a
    printf("c = %d \n", c);
    c %= a; // c = c%a
    printf("c = %d \n", c);
    return 0;
}
```

```
c = 5
c = 10
c = 5
c = 25
c = 5
c = 0
```

Relational operators

- These operators are used to form relational expressions, which evaluates to either true or false. (True - 1, false - 0).
- It is used for comparisons. C supports six relational operators.

Operator	Meaning
<	is less than
<=	is less than or equal to
>	is greater than
>=	is greater than or equal to
==	is equal to
!=	is not equal to

```c
#include <stdio.h>
int main()
{
    int a = 5, b = 5, c = 10;

    printf("%d == %d = %d \n", a, b, a == b); // true
    printf("%d == %d = %d \n", a, c, a == c); // false

    printf("%d > %d = %d \n", a, b, a > b); //false
    printf("%d > %d = %d \n", a, c, a > c); //false


    printf("%d < %d = %d \n", a, b, a < b); //false
    printf("%d < %d = %d \n", a, c, a < c); //true


    printf("%d != %d = %d \n", a, b, a != b); //false
    printf("%d != %d = %d \n", a, c, a != c); //true


    printf("%d >= %d = %d \n", a, b, a >= b); //true
    printf("%d >= %d = %d \n", a, c, a >= c); //false


    printf("%d <= %d = %d \n", a, b, a <= b); //true
    printf("%d <= %d = %d \n", a, c, a <= c); //true

    return 0;
}
```

```
5 == 5 = 1
5 == 10 = 0
5 > 5 = 0
5 > 10 = 0
5 < 5 = 0
5 < 10 = 1
5 != 5 = 0
5 != 10 = 1
5 >= 5 = 1
5 >= 10 = 0
5 <= 5 = 1
5 <= 10 = 1
```

Bitwise operators

- These operators which are used for the manipulation of data at bit level are known as bitwise operators.
- These are used for testing the bits or shifting them right or left.

Operator	Meaning
&	Binary (bitwise) AND
|	Bitwise OR
^	Bitwise exclusive OR
<<	Bitwise left shift
>>	Bitwise right shift
~	Bitwise Ones Complement

Bitwise AND operator &

- The output of bitwise AND is 1 if the corresponding bits of two operands is 1.
- If either bit of an operand is 0, the result of corresponding bit is evaluated to 0.

Bitwise OR operator |

- The output of bitwise OR is 1 if at least one corresponding bit of two operands is 1. In C Programming, bitwise OR operator is denoted by |

```
12 = 00001100 (In Binary)
25 = 00011001 (In Binary)

Bitwise OR Operation of 12 and 25
   00001100
|  00011001
   ________
   00011101   = 29 (In decimal)
```

```c
#include <stdio.h>
int main()
{
    int a = 12, b = 25;
    printf("Output = %d", a|b);
    return 0;
}
```

A	B	A&B	A|B	~A
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

Table : Truth Table for bitwise AND, OR and Complement

Bitwise AND

```
12 = 00001100 (In Binary)
25 = 00011001 (In Binary)

Bit Operation of 12 and 25
  00001100
& 00011001

  _________
  00001000  = 8 (In decimal)
```

```c
#include <stdio.h>
int main()
{
    int a = 12, b = 25;
    printf("Output = %d", a&b);
    return 0;
}
```

Bitwise XOR

```
12 = 00001100 (In Binary)
25 = 00011001 (In Binary)

Bitwise XOR Operation of 12 and 25
  00001100
^ 00011001

  _________
  00010101  = 21 (In decimal)
```

```c
#include <stdio.h>
int main()
{
    int a = 12, b = 25;
    printf("Output = %d", a^b);
    return 0;
}
```

Right Shift Operator

```
212 = 11010100 (In binary)
212>>2 = 00110101 (In binary) [Right shift by two bits]
212>>7 = 00000001 (In binary)
212>>8 = 00000000
212>>0 = 11010100 (No Shift)
```

Left Shift Operator

```
212 = 11010100 (In binary)
212<<1 = 110101000 (In binary) [Left shift by one bit]
212<<0 = 11010100 (Shift by 0)
212<<4 = 110101000000 (In binary) =3392(In decimal)
```

Shift Operators

```c
#include <stdio.h>
int main()
{
    int num=212, i;
    for (i=0; i<=2; ++i)
        printf("Right shift by %d: %d\n", i, num>>i);

     printf("\n");

     for (i=0; i<=2; ++i)
        printf("Left shift by %d: %d\n", i, num<<i);

     return 0;
}
```

o/p

```
Right Shift by 0: 212
Right Shift by 1: 106
Right Shift by 2: 53

Left Shift by 0: 212
Left Shift by 1: 424
Left Shift by 2: 848
```

Increment and Decrement Operators

- The increment operator (++) increases its operand by 1 and the decrement (--) decreases its operand by 1.
- They are the unary operators (i.e. they operate on a single operand). It can be written as:
- x++ or ++x

(post and pre increment)

- x--or --x

(post and pre decrement)

```c
#include <stdio.h>
int main()
{
    int a = 10, b = 100;
    float c = 10.5, d = 100.5;

    printf("++a = %d \n", ++a);

    printf("--b = %d \n", --b);

    printf("++c = %f \n", ++c);

    printf("--d = %f \n", --d);

    return 0;
}
```

```
++a = 11
--b = 99
++c = 11.500000
--d = 99.500000
```

Conditional Operator

- The ternary operator pair ? and : are used to construct conditional operator.
- An expression that makes use of conditional operator is called conditional expression.
- Its general syntax is:

Condition? TRUE case : FALSE case;

- **Example:** x=(a>b)?a:b;

```c
#include <stdio.h>
int main(){
    char February;
    int days;
    printf("If this year is leap year, enter 1. If not enter any integer: ");
    scanf("%c",&February);

    // If test condition (February == 'l') is true, days equal to 29.
    // If test condition (February =='l') is false, days equal to 28.
    days = (February == '1') ? 29 : 28;

    printf("Number of days in February = %d",days);
    return 0;
}
```

```
If this year is leap year, enter 1. If not enter any integer: s
Number of days in February = 28
```

Operator Precedence

- Operator precedence determines the grouping of terms in an expression and decides how an expression is evaluated.
- Certain operators have higher precedence than others; for example, the multiplication operator has a higher precedence than the addition operator.
- For example, x = 7 + 3 * 2; here, x is assigned 13, not 20 because operator * has a higher precedence than +, so it first gets multiplied with 3*2 and then adds into 7.

Priority

- This represents the evaluation of expression starts from "what" operator.

Associativity

- It represents which operator should be evaluated first if an expression is containing more than one operator with same priority.

Operator	Priority	Associativity
{}, (), []	1	Left to right
++, --, !	2	Right to left
*, /, %	3	Left to right
+, -	4	Left to right
<, <=, >, >=, ==, !=	5	Left to right
&&	6	Left to right
||	7	Left to right
?:	8	Right to left
=, +=, -=, *=, /=, %=	9	Right to left

Type casting

- When an operator is used to manipulate operands of different data type then each operand should have same data type to take operation because the final result must be in one data type.
- For example, the given expression is: **z = int x + float y**; then the data type of z must be only one.
- To handle such type of problems, the type conversion and type casting are introduced.
- The process of converting one data type into another type is known as type casting.
- C provides two types of type conversions:
  - ✓**Implicit type conversion:**
  - ✓**Explicit type conversion:**

Implicit type conversion:

- In implicit type conversion, if the operands of an expression are of different types, the lower data type is automatically converted to the higher data type before the operation evaluation.
- The result of the expression will be of higher data type.
- it is the automatic type conversion done by the compiler.

Explicit type conversion:

- In explicit type conversion, the user has to enforce the compiler to convert one data type to another data type by using typecasting operator.
- This method of typecasting is done by prefixing the variable name with the data type enclosed within parenthesis.
- **Syntax:**

(data type) variable/expression/value;

Example:

```c
#include<stdio.h>
int main()
{
    float  sum, sum1;
sum  =  (int)  (1.5  *  3.8);
sum1 = 1.5*3.8;
printf("%.1f",sum);
printf("\n%f",sum1);
}
```

```
H:\My Drive\Cprogram slides program\inputoutput\typecast.exe
5.0
5.700000
--------------------------------
Process exited after 0.05303 seconds with return value 0
Press any key to continue . . .
```

The typecast (int) tells the C compiler to interpret the result of (1.5 * 3.8) as the integer 5, instead of 5.7. Then, 5.0 will be stored in sum, because the variable sum is of type float.

- Data input and Output

Input/Output functions

- When we say Input, it means to feed some data into a program.
- An input can be given in the form of a file or from the command line.
- C programming provides a set of built-in functions to read the given input and feed it to the program as per requirement.
- When we say Output, it means to display some data on screen, printer, or in any file.
- C programming provides a set of built-in functions to output the data on the computer screen as well as to save it in text or binary files.

Formatted Input Output Functions

- Formatted input output functions accept or present the data in a particular format.
- The standard library consists of different functions that perform output and input operations.
- Out of these functions, printf() and scanf() allow user to format input output in desired format.
- printf() and scanf() can be used to read any type of data (integer, real number, character etc.).

printf() Library Function with Examples

- printf() is formatted output function which is used to display some information on standard output unit.
- It is defined in standard header file stdio.h. So, to use printf(), we must include header file stdio.h in our program.

Syntax for printf() is :

- printf("String");

  or

- printf(“format_string”, var1, var2, var3, …, varN);

Where format_string may contain :

- Characters that are simply printed as they are.
- Format specifier that begin with a % sign.
- Escape sequences that begin with \ sign.

Format Specifier (%) in printf()

```
#include<stdio.h>
void main()
{
        int a;
        float b;
        a=2;
        b=4.5;
        printf("Value of a = %d and b = %f.", a, b);
}
```

Escape Sequences (\ ) in printf()

```
#include<stdio.h>
void main()
{
        printf("Hello, World!");
        printf("\nNow control is on new line and this \t prints tab.");
}
```

scanf() Library Function with Examples

- scanf() is formatted input function which is used to read formatted data from standard input device and automatically converts numeric information to integers and floats.
- It is defined in stdio.h.

scanf() Syntax:

- scanf("format_string", &var1, &var2, &var3, ..., &varN);
- & in above syntax is called address operator.

Reading Integer and Floating Point Number & Displaying Them

```
#include<stdio.h>
void main()
{
  int a;
  float b;
  printf (“Enter value of a and b: ”);
  scanf(“%d%f”, &a, &b);
  printf(“Value of a = %d \n Value of b = %f”, a, b);
}
```

Unformatted Input Output Functions

- Unformatted input output functions cannot control the format of reading and writing the data.
- These functions are the most basic form of input and output and they do not allow to supply input or display output in user desired format that's why we call them unformatted input output functions.
- Unformatted input output functions are classified into two categories as character input output functions and string input output functions.
- Character input functions are used to read a single character from the keyboard and character output functions are used to write a single character to a screen.
- getch(), getche(), and getchar() are unformatted character input functions while putch() and putchar() are unformatted character output functions.

- different unformatted character input output functions

getch() Library Functions with Examples

- getch() is character input functions. It is unformatted input function meaning it does not allow user to read input in their format.
- It reads a character from the keyboard but does not echo the pressed character and returns character pressed.
- It is defined in header file conio.h.

getch() Syntax

- character_variable = getch();

Reading Character Using getch() & Displaying

```
#include<stdio.h>
#include<conio.h>

void main()
{
  char ch;
  printf("Press any character: ");
  ch = getch();
  printf("\nPressed character is: %c", ch);
}
```

getche() Library Functions with Examples

- Like getch(), getche() is also character input functions.
- It is unformatted input function meaning it does not allow user to read input in their format.
- Difference between getch() and getche() is that getche() echoes pressed character.
- getche() also returns character pressed like getch().
- It is also defined in header file conio.h.

getche() Syntax

- character_variable = getche();

Reading Character Using getche() & Displaying

```
#include<stdio.h>
#include<conio.h>

void main()
{
  char ch;
  printf(“Press any character: ”);
  ch = getche();
  printf(“\nPressed character is: %c”, ch);
}
```

getchar() Library Functions with Examples

- getchar() is also a character input functions.
- It reads a character and accepts the input until carriage return is entered (i.e. until enter is pressed).
- It is defined in header file stdio.h.

getchar() Syntax

- character_variable = getchar();

Reading Character Using getchar() & Displaying

```
#include<stdio.h>
void main()
{
  char ch;
  printf(“Press any character: ”);
  ch = getchar();
  printf(“\nPressed character is: %c”, ch);
}
```

putch() Library Function in C with Examples

- The putch() function is used for printing character to a screen at current cursor location.
- It is unformatted character output functions.
- It is defined in header file conio.h.

putch() Syntax

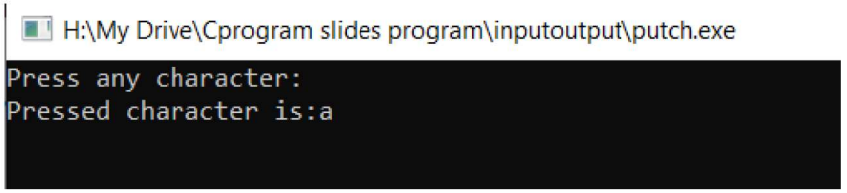
- putch(character);

  or

- putch(character_variable);

Displaying Character Using putch()

```
#include<stdio.h>
#include<conio.h>
void main()
{
  char ch;
  clrscr(); /* Clear the screen */
  printf("Press any character: ");
  ch = getch();
  printf("\nPressed character is:");
  putch(ch);
  getch(); /* Holding output */
}
```



putchar() Library Function with Examples

- The putchar() function is used for printing character to a screen at current cursor location.
- It is unformatted character output functions.
- It is defined in header file stdio.h.

putchar() Syntax

- putchar(character);

  or

- putchar(character_variable);

Displaying Character Using putchar()

```
#include<stdio.h>
#include<conio.h>
void main()
{
  char ch;
//  clrscr(); /* Clear the screen */
  printf("Press any character: ");
  ch = getche();
  printf("\nPressed character is:");
  putchar(ch);
  getch(); /* Holding output */
}
```

gets() and puts() functions in C

- In programming, collection of character is known as string or character array.
- In C programming, there are different input output functions available for reading and writing of strings.
- Most commonly used are gets() and puts().
- gets() is unformatted input function for reading string and puts() is unformatted output function for writing string.

gets() Library Functions with Examples

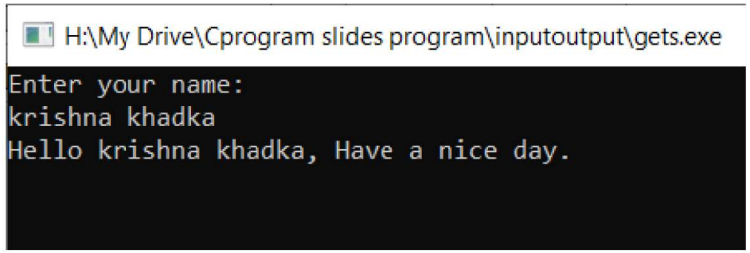
- gets() is string input functions. It reads string from the keyboard. It reads string until enter is pressed. It is defined in header file stdio.h.

gets() Syntax

- gets(string_variable);

Reading String Using gets() Function

```
#include<stdio.h>
#include<conio.h>
void main()
{
  char name[100];
  printf(“Enter your name:\n”);
  gets(name);
  printf("Hello %s, Have a nice day.", name);
  getch(); /* Holding output */
}
```



puts() Library Function with Examples

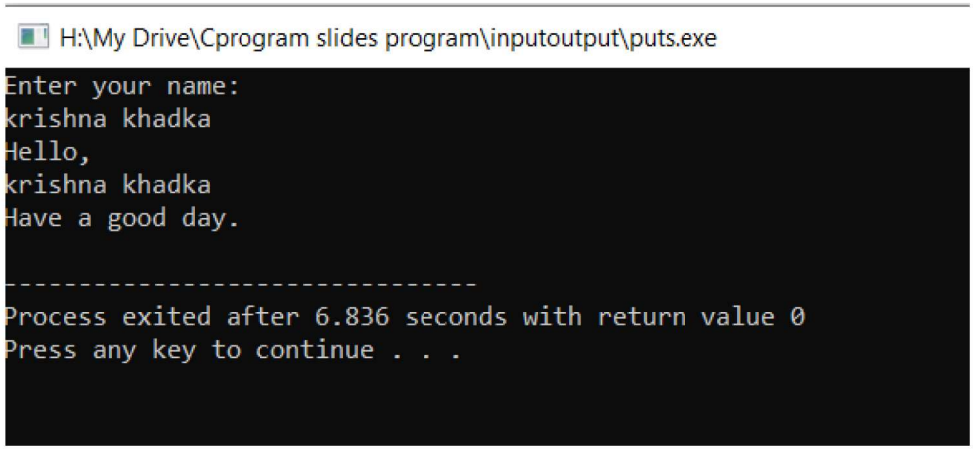
- puts() is unformatted string output functions.
- It writes or prints string to screen.
- It is defined in standard header file stdio.h.

puts() Syntax

- puts(string or string_variable);

Displaying String Using puts() Function

```
#include<stdio.h>
#include<conio.h>
void main()
{
  char name[100];
  printf("Enter your name:\n");
  gets(name);
  puts("Hello,");
  puts(name);
  puts("Have a good day.");
}
```



Difference between Formatted and Unformatted Functions

- **Formatted** I/O functions allow to supply input or display output in user desired format.
- **Unformatted** I/O functions are the most basic form of input and output and they do not allow to supply input or display output in user desired format.
- printf() and scanf() are examples for formatted input and output functions.
- getch(), getche(), getchar(), gets(), puts(), putchar() etc. are examples of unformatted input output functions.
- **Formatted** input and output functions contain format specifier in their syntax.
- **Unformatted** input and output functions do not contain format specifier in their syntax.
- **Formatted** I/O functions are used with all data types.
- **Unformatted** I/O functions are used mainly for character and string data types.

- Thank you